

Lab Task 3: Word Generation using LSTM

Aim:

To analyze word generation using a Single Layer LSTM (Long Short-Term Memory) model with preprocessing using NLTK and spaCy in Python.

CODE:

```
# Install (run once)

# !pip install numpy nltk tensorflow

import numpy as np

import nltk

from nltk.tokenize import word_tokenize

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense, Embedding

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.utils import to_categorical

# Download tokenizer

nltk.download('punkt')

nltk.download('punkt_tab')

# Sample text

text = """Natural language processing enables computers to understand human language.
It is a key area of artificial intelligence."""

# ◆ Preprocessing

tokens = word_tokenize(text.lower())

# Vocabulary

vocab = sorted(set(tokens))

word_to_index = {word: i for i, word in enumerate(vocab)}

index_to_word = {i: word for word, i in word_to_index.items()}

# ◆ Create sequences

sequences = []

for i in range(1, len(tokens)):

    seq = tokens[:i+1]
```

```

    sequences.append([word_to_index[word] for word in seq])
# Padding
max_len = max(len(seq) for seq in sequences)
sequences = pad_sequences(sequences, maxlen=max_len, padding='pre')
# Split into X and y
X = sequences[:, :-1]
y = sequences[:, -1]
# One-hot encoding
y = to_categorical(y, num_classes=len(vocab))
# ♠ Build LSTM Model
model = Sequential()
model.add(Embedding(input_dim=len(vocab), output_dim=10, input_length=max_len-1))
model.add(LSTM(100))
model.add(Dense(len(vocab), activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
# Train
model.fit(X, y, epochs=200, verbose=0)
# ♠ Text Generation Function
def generate_text(seed_text, next_words):
    for _ in range(next_words):
        token_list = [word_to_index.get(word, 0) for word in seed_text.lower().split()]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')
        predicted = np.argmax(model.predict(token_list, verbose=0), axis=-1)[0]
        output_word = index_to_word[predicted]
        seed_text += " " + output_word
    return seed_text
# Output
print("Generated Text:")
print(generate_text("natural language", 5))

```

Output:

Generated Text:

natural language processing enables computers to understand